

Day 104: Linear Regression

1. Day Number

Day 104

2. Topic Name

Supervised Learning and Linear Regression

3. Connection

You already understand **features, labels, training/test data, overfitting, feature engineering, and gradients**.

Today you will connect those ideas by working toward your **first simple regression model**.

The basic idea is:

Feature → Linear Regression Model → Numerical Prediction

For example:

House size → Model → Predicted house price

4. Important Topics

Supervised learning means the model learns from examples containing both input features and known target values.

Regression is supervised learning where the target is a numerical value, often continuous.

Examples include predicting:

house price
temperature
monthly sales
delivery time

A **continuous target** can take many numerical values rather than being limited to categories such as "yes/no."

A **line of best fit** is a line that tries to represent the general relationship between the input and target.

A **coefficient** tells us how strongly the prediction changes when a feature changes.

An **intercept** represents the model's starting/base value when the input feature is zero.

A **prediction** is the estimated target produced by the trained model.

5. Foundational Notes

Suppose you have this tiny dataset:

House Size	House Price
1000	200
1500	300
2000	400
2500	500

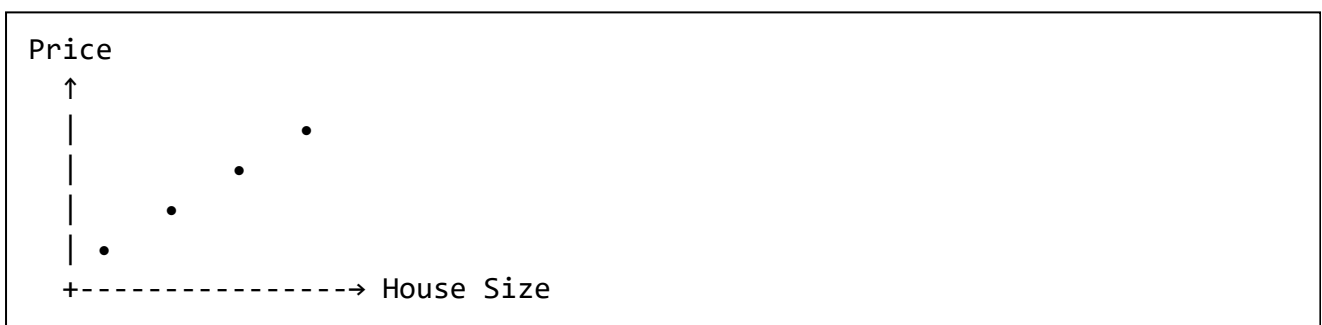
Here:

Feature (X) = House Size
Target (y) = House Price

Because the target is numerical, this is a **regression problem**.

Linear Regression tries to learn a straight-line relationship between them.

Conceptually:



The model tries to place a line through the data that represents the overall pattern.

6. Connecting Linear Regression With $y = mx + b$

You may remember the equation of a straight line:

$$y = mx + b$$

In simple Linear Regression, the same idea appears as:

prediction = coefficient $\times$ feature + intercept

So:

$$y = m \times x + b$$

can be interpreted as:

y = predicted target
 x = input feature
 m = coefficient
 b = intercept

Suppose a hypothetical model learned:

$\text{Price} = 0.2 \times \text{Size} + 10$

For a house of size 1500:

Price prediction
 $= 0.2 \times 1500 + 10$

You do not need to calculate this manually when using scikit-learn. The library learns the coefficient and intercept from the training data.

7. Easy Example: House Size → House Price

Imagine:

Size	Price
1000	210
1200	245
1500	295
1800	350
2000	390

You might notice:

┆ Larger houses generally have higher prices.

Linear Regression tries to summarize that relationship with a line.

The training process conceptually looks like:

Known house sizes
+
Known house prices
↓
Linear Regression
↓
Learn coefficient + intercept
↓
Use line for new predictions

The individual data points do **not** have to sit perfectly on the line.
Real data usually contains variation.

8. Problem Statement

Use this tiny dataset:

Study Hours	Exam Score
1	52
2	58
3	65
4	72
5	78

Your task is to build a basic Linear Regression model where:

Feature = Study Hours
Target = Exam Score

Then use the trained model to predict the exam score for:

Study Hours = 6

Use **scikit-learn**, but do not worry about optimizing the model.

The purpose is to understand the training and prediction workflow.

9. Concepts Used

This exercise combines concepts from several previous days:

```
Dataset
  ↓
Feature + Target
  ↓
Supervised Learning
  ↓
Regression
  ↓
Train Model
  ↓
Learn Coefficient + Intercept
  ↓
Prediction
```

You are also connecting your earlier gradient lesson to model training.

During training, learning algorithms try to find parameters that reduce prediction error. You do not need to implement gradient descent yourself for today's exercise.

10. Thought Process

First identify the machine-learning problem.

We want to predict:

Exam Score

using:

Study Hours

Therefore:

$X = \text{Study Hours}$
 $y = \text{Exam Score}$

Next ask what type of target Exam Score is.

It is numerical.

Therefore this is a:

regression problem

Then choose a simple regression model:

Linear Regression

Give the training examples to the model.

The model learns approximately:

$\text{Score} = \text{coefficient} \times \text{Study Hours} + \text{intercept}$

Finally, provide a new input:

6 study hours

and ask the model for a prediction.

11. Beginner-Friendly Pseudocode

```
START

import the Linear Regression tool

create study-hours data
create exam-score data

prepare the feature in the shape expected by the model

create a Linear Regression model

train the model using:
    study hours
    exam scores

create new input:
    6 study hours

ask model to predict the score

print prediction

optionally inspect:
    coefficient
    intercept

END
```

Notice the two important operations:

```
fit
```

means:

| Learn from the training examples.

And:

```
predict
```

means:

| Use what was learned to estimate a target for new input.

12. Suggested Solving Approach: scikit-learn

For today's exercise, use the **LinearRegression** model from scikit-learn.

Your program will conceptually need:

```
from sklearn.linear_model import LinearRegression
```

Then think about this sequence:

```
Prepare X
Prepare y
  ↓
Create model
  ↓
model.fit(...)
  ↓
model.predict(...)
```

One important beginner detail is that scikit-learn usually expects the feature data X to be two-dimensional.

Conceptually, instead of:

```
[1, 2, 3, 4, 5]
```

think:

```
[[1],
 [2],
 [3],
 [4],
 [5]]
```

Each inner row represents one observation containing one feature.

Try to construct the actual Python code yourself rather than copying a complete solution.

13. Easy Edge Cases

Tiny Dataset

Suppose you train using only two houses.

The model can still produce a line, but you should not automatically trust it.

Very little data gives us limited evidence about the real relationship.

Unusual Input

Suppose the training houses range from:

```
1000–2500 square feet
```

and then you ask the model to predict:

10000 square feet

The model can mathematically produce a prediction, but that prediction may be unreliable because the new input is far outside the range it learned from.

Predicting beyond the observed range is called **extrapolation**.

No Clear Linear Relationship

Not every relationship forms something close to a straight line.

If the true relationship is very different, simple Linear Regression may perform poorly.

Duplicate Feature Values

You could have:

2 study hours → score 55
2 study hours → score 61

That is not automatically an error.

Real-world outcomes can differ even when one feature is the same because other factors may also matter.

14. Expected Prediction Flow

Your exercise should conceptually produce this sequence:

```
Study Hours + Known Scores
    ↓
Prepare X and y
    ↓
Create LinearRegression model
    ↓
Train with fit()
    ↓
Model learns:
coefficient + intercept
    ↓
Give new input:
6 study hours
    ↓
predict()
    ↓
Predicted exam score
```

You can also inspect the trained model conceptually as:

learned coefficient:
How much the predicted score tends to change
when study hours increase by one unit.

learned intercept:
The model's estimated starting point when
study hours = 0.

Do not assume the coefficient proves that additional study **causes** the score increase. Linear Regression here is learning an association from the provided data.

15. Hint Only

Start by separating the tiny table into:

```
X = study hours  
y = exam scores
```

Remember that X should look roughly like:

```
[[1],  
 [2],  
 [3],  
 [4],  
 [5]]
```

Then think about these three scikit-learn steps:

```
create model  
  ↓  
fit model  
  ↓  
predict
```

For the final prediction, your new feature must have the same basic structure as the training features.

The main lesson for **Day 104** is:

```
Known feature-target examples  
  ↓  
Linear Regression learns a line  
  ↓  
New feature value  
  ↓  
Numerical prediction
```

You do **not** need to manually calculate the best-fit line or implement gradient descent yet.